CSC 4553

Fall 2026 - Day 14

Admin

- Lab due Friday, 18:00
- Quiz due Monday, 18:00
- Sign up for code review slots
  https://calendar.app.google/XGVki6gTCicVj8BU9
- Mid-semester survey on Canvas

Deadlocks

Questions to consider

- What are the four necessary conditions for deadlock, and why must all four be present simultaneously?
- How can Resource Allocation Graphs help us visualize and understand when deadlock might occur?
- Why is a cycle necessary but not sufficient for deadlock when there are multiple instances of a resource?

Deadlocks

- If we're not careful about dependencies, we might create pathologies that lead to deadlock
  - → Like race conditions, may only happen sometimes, making them hard to identify and debug
- Example: Three processes, three resources of the same type, each process requires two resources to make progress
  - → This can happen across a network as well, which is particularly hard to diagnose
- Can we **detect and resolve** these situations? Can we **prevent** them?

Deadlock example

- Classic example: Thread1 holds lock L1 and is waiting for lock L2. Thread2 holds L2 and is waiting for L1

```
Thread 1:                       Thread 2:
pthread_mutex_lock(L1);         pthread_mutex_lock(L2);
pthread_mutex_lock(L2);         pthread_mutex_lock(L1);
```

Deadlock characteristics

- Definition: A set of processes are deadlocked if each process in the set is waiting for an event that only another process in the set can cause.
  - → **Resource deadlock** is the most common (and what we'll focus on)

Conditions for deadlock

- Four necessary conditions (simultaneous):
  - → **Mutual exclusion**: At least one resource can only be held by one process at a time; this can result in other processes waiting for that resource
  - → **Hold and wait**: A process must be holding at least one resource while waiting for other resources (held by other processes)
  - → **No preemption**: Resources can only be released voluntarily by a process; Resources cannot be revoked
  - → **Circular wait**: A set of n waiting processes $\{P_0, \dots, P_{n-1}\}$ such that P_i is waiting for resources held by $P_{i+1 \bmod n}$
- All must be met

Deadlock conditions are design choices

Condition	The question to ask	Answer that allows deadlock
Mutual exclusion	Can two processes use this resource at the same time?	No: one owner at a time (e.g., a mutex)
Hold and wait	Can a process keep what it has while it waits for more?	Yes: grab one lock, then wait for the next
No preemption	Can the OS take a resource away from a process?	No: only the owner can give it back
Circular wait	Can processes request resources in any order?	Yes: T1 locks A then B, T2 locks B then A

- Each condition exists because of a rule the system chose. Pick a different rule, and the condition disappears.
- Change the answer to **any one** question and deadlock becomes impossible

Deadlock modeling

- Resource Allocation Graph: a directed graph; processes (circles) and resources (squares) are nodes
  - → Request edge: thread → resource
  - → Assignment edge: resource → thread
- Cycles in a graph indicate possible deadlock
  - → One instance per resource: a cycle is necessary and sufficient
  - → **Multiple instances: a cycle is necessary but not sufficient**

Example: the bank transfer

```
1 void transfer(account *from, account *to, int amt) {
2   pthread_mutex_lock(&from->lock);
3   pthread_mutex_lock(&to->lock);
4   from->balance -= amt;
5   to->balance   += amt;
6   pthread_mutex_unlock(&to->lock);
7   pthread_mutex_unlock(&from->lock);
8 }
```

```
Thread 1: transfer(&alice, &bob, 50);
Thread 2: transfer(&bob, &alice, 20);
```

- **Think**: Can this deadlock? Write down an interleaving that causes it.
- **Discuss**: Together, draw the RAG at the moment it's stuck. Find the line(s) of code responsible for **each** of the four conditions.

Push it further

With others, decide **deadlock or not** for each, and justify with the four conditions or the RAG:

1. Three threads: `transfer(A,B)`, `transfer(B,C)`, `transfer(C,A)`
2. One thread: `transfer(&alice, &alice, 10)`
3. What is the **smallest** change to `transfer()` that makes the original two-thread case safe?

What isn’t clear?

Comments? Thoughts?

Handling deadlocks

Questions to consider

- What are the approaches to handling deadlock? How practical is each? How do they differ in approach?
- How does deadlock avoidance differ from deadlock prevention? What techniques do we implement for deadlock avoidance?
- What are the trade-offs between detecting/recovering from deadlock versus preventing or avoiding it, and when is detection practical?

How should the OS handle deadlocks?

- OS can **prevent** deadlock: ensure deadlocks can never occur (by preventing one of the deadlock requirements from occurring)
- OS can **avoid** deadlock: like prevention, but given more information about a process's resource needs
- OS can **detect** deadlock: allow system to deadlock, detect and recover
- OS can **do nothing**, and let applications handle it
  - → **Ostrich Algorithm**: head in sand; wait

Deadlock prevention

- Remember, we just have to break one of these to prevent deadlock
  - → Mutual exclusion
  - → Hold and wait
  - → No preemption
  - → Circular wait
- Any ideas?

Attacking mutual exclusion

- Make resources sharable so they aren't held by only one process at a time; some resources are naturally sharable (e.g., read-only files), but preventing mutual exclusion is fundamentally difficult as so many resources are non-sharable: printers, writable files, mutex locks, etc.

  → Spoolers can sidestep this for some resources (e.g., a print spooler accepts jobs so no one "owns" the printer directly)

Attacking hold and wait

- When a process requests a resource, it cannot be holding any other resources
- A process must request **all** of its resources at one time
- Example, treat group of lock acquisitions like a critical section and surround with a mutex

```
1 pthread_mutex_lock(global_lock);
2 pthread_mutex_lock(L1);
3 pthread_mutex_lock(L2);
4 ...
5 pthread_mutex_unlock(global_lock);
```

- See any downsides?
  - → Must know all needed locks a priori
  - → Will likely decrease concurrency

Attacking no preemption

- If a process is holding some resources and requests another that cannot be immediately allocated, then all resources held are released
- Process is restarted only when it can regain its old resources plus the new
- Downsides?
  - → Not all resources are preempt-able
  - → Starvation is possible

Attacking circular wait

- How could you prevent by focusing on the circular wait requirement?
- Impose a total ordering of all resource types
  - → For example, resource types R1, R2, …, Rn have ordering R1 < R2 < … < Rn
  - → A process that needs multiple resources must request them in increasing order of enumeration
- Downsides?
  - → May be difficult to impose a meaningful ordering
  - → May lead to reduced concurrency

Fix the transfer

```
1 void transfer(account *a, account *b, int amt) {
2   pthread_mutex_lock(&a->lock);
3   pthread_mutex_lock(&b->lock);
4   a->balance -= amt;
5   b->balance += amt;
6   pthread_mutex_unlock(&b->lock);
7   pthread_mutex_unlock(&a->lock);
8 }
```

- **Think**: Rewrite `transfer()` so it can never deadlock. Which condition did you break?
- **Discuss**: Compare with a neighbor. Then extend it: `transfer_many(account *accts[], int n)` must lock all `n` accounts before moving money.

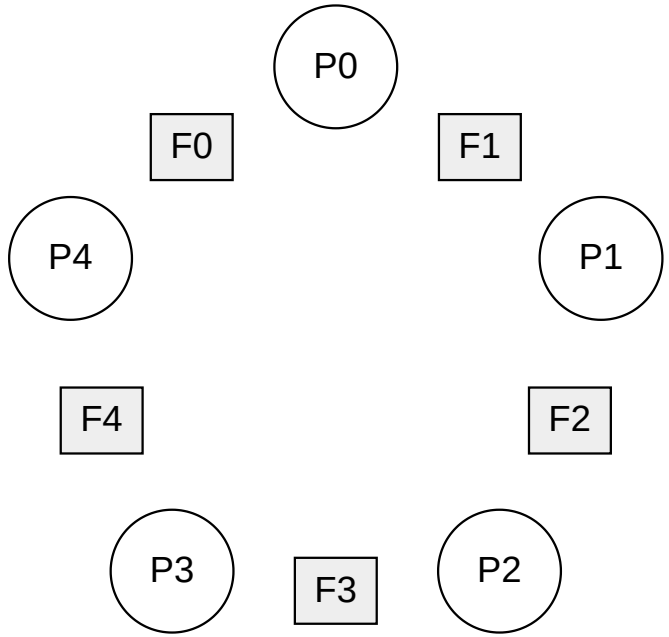
Deadlock prevention summary

- Which approach is best?
  - → Attacking mutual exclusion is generally not feasible
  - → Attacking hold and wait is wasteful and requires prescience
  - → Attacking no preemption is difficult due to non-preemptable resources
  - → Attacking circular wait is practical but may reduce concurrency - often the best choice and **most used**

Dining philosophers

- Five philosophers around a table, one fork between each pair
- To eat, a philosopher needs **both** adjacent forks

```
1  void philosopher(int i) {
2    while (1) {
3      think();
4      lock(fork[i]);             // left
5      lock(fork[(i + 1) % 5]);   // right
6      eat();
7      unlock(fork[(i + 1) % 5]);
8      unlock(fork[i]);
9    }
10 }
```



Break one condition

- **Think**: How can the philosophers deadlock? Point to each of the four conditions in the code.
- **Discuss**: For each condition, explain how it can be broken. Change the code (or the rules) so deadlock is impossible.
  - → Mutual exclusion
  - → Hold and wait
  - → No preemption
  - → Circular wait
- Also consider the **cost** of your fixes

```
1  void philosopher(int i) {
2    while (1) {
3      think();
4      lock(fork[i]);          // left
5      lock(fork[(i+1) % 5]); // right
6      eat();
7      unlock(fork[(i+1) % 5]);
8      unlock(fork[i]);
9    }
10 }
```

What isn’t clear?

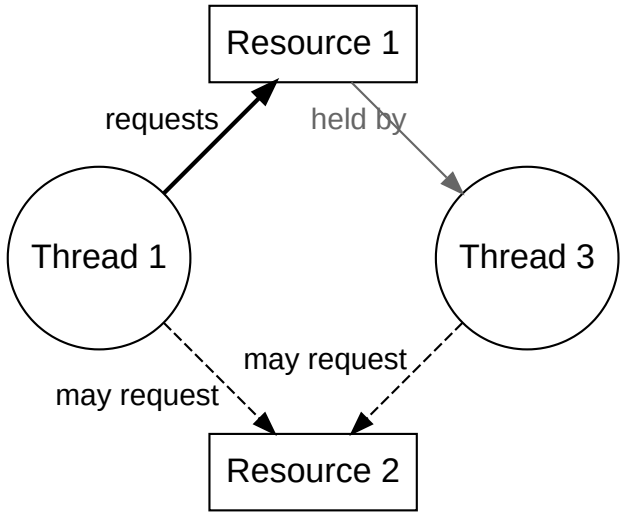
Comments? Thoughts?

Deadlock avoidance

- All of the prevention techniques can be heavy-handed, reducing concurrency and overall system throughput
- What if we had more information about process resource needs before allocation?
  - → Could we make smarter decisions about resource allocation?
- This is the idea behind deadlock avoidance

Avoidance via resource-allocation graph

- Introduce the idea of a **claim edge** (dashed line) from a process to a resource
  - → Indicates that the process may request that resource in the future
- The system can maintain this graph, and only grant resources that will not create a cycle
- Any downsides / limitations?
  - → Overhead of maintaining the graph and checking for cycles
  - → Only applicable to single instance resources



Deadlock avoidance in practice

- Dijkstra's Banker's algorithm is the textbook example — each process declares its max resource needs, and requests are only granted if the resulting state is "safe." Not used in practice: overhead plus the prescience requirement
- Resource-allocation graph with claim edges sees occasional use in constrained settings (e.g., embedded systems where max needs are predictable)

Detecting and recovering from deadlock

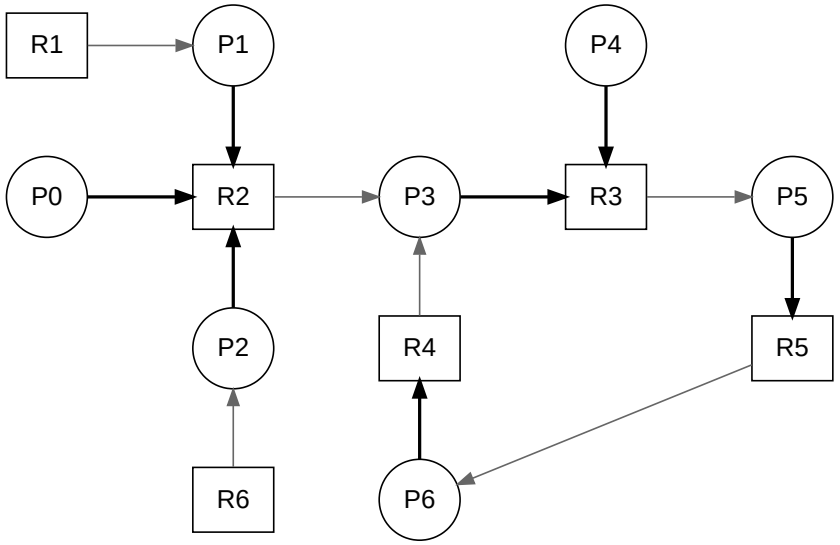
- Allow deadlocks to occur, then detect and recover
- In some cases, recovery is pretty simple. If your machine froze once per year (maybe it does?) what would you do?
  - → Reboot
- What scenarios do you think complex detection and recovery would make sense?
  - → Systems with high availability requirements where manual intervention is costly or impractical
    - telecommunications systems, financial transaction systems, healthcare systems
    - critical infrastructure systems

Detection

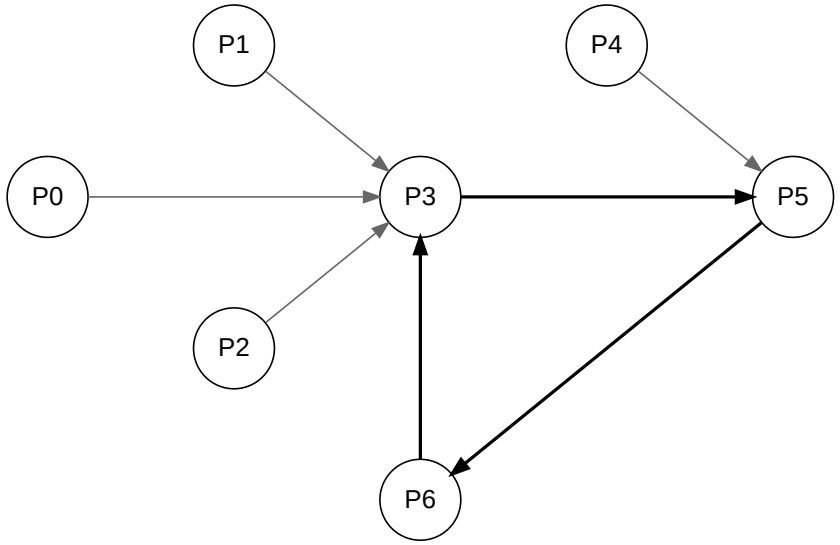
- Use a wait-for graph: a simplified resource-allocation graph that removes resource nodes
  - → An edge from process P_i to P_j indicates that P_i is waiting for a resource held by P_j
  - → A cycle in this graph indicates a deadlock
  - → $O(n^2)$ algorithm to search for cycles (beyond the cost of maintaining the graph itself)
  - → Only applies to single instance resources

Detection (cont'd)

Resource-allocation graph



Wait-for graph



Detection (cont'd)

- For multiple instance resources, we can use an algorithm similar to the Banker's algorithm to detect deadlock
- Question: How often should we run the detection algorithm?
  - → Too often: wasteful overhead
  - → Not often enough: long delays before recovery
  - → What if we run it when a process requests a resource?
    - Could allow us to assign blame for deadlock to the requesting process
  - → Heuristically (CPU utilization, etc.)
    - Less overhead, but harder to assign blame

Recovery options

- Process termination
  - → Abort all deadlocked processes (brute force)
  - → Abort one process at a time until deadlock is resolved
    - How to choose which process to abort?
      - Priority, time running, resources used, etc.
- Resource preemption
  - → Temporarily take resources away from some processes and give to others until deadlock is resolved
  - → Highly dependent on the resource type
  - → Rollback problem - processes may need to be rolled back to a safe state before resources can be reallocated
  - → Starvation problem - same victims may be chosen repeatedly

Pick the policy

- For each system below, which deadlock strategy fits best — and **why**?
  1. Flight control software on a commercial aircraft
  2. A university course-registration system during peak enrollment
  3. Your personal laptop running a browser, editor, and Slack
- For your pick, name the cost you're accepting (lost concurrency? overhead? user-visible failures?)

What isn’t clear?

Comments? Thoughts?